CEUE203
25IT037

PRACTICAL :11

Program - 1: Save and Load Objects Using Serialization

Aim:

To implement object serialization and deserialization in Java using Serializable, ObjectOutputStream and ObjectInputStream, and demonstrate the use of the transient keyword.

Code:

```
import java.io.*;


class Student implements Serializable {
    int id;
    String name;
    transient String password;


    Student(int id, String name, String password) {
        this.id = id;
        this.name = name;
        this.password = password;
    }
}


public class SaveLoadDemo {
    public static void main(String[] args) {
        try {
```

```
        Student[] students = {
            new Student(1, "Rahul", "1234"),
            new Student(2, "Amit", "5678")
        };

        ObjectOutputStream out =
            new ObjectOutputStream(
                new FileOutputStream("student.dat"));

        out.writeObject(students);
        out.close();

        System.out.println("Objects saved successfully.");

        ObjectInputStream in =
            new ObjectInputStream(
                new FileInputStream("student.dat"));

        Student[] loaded =
            (Student[]) in.readObject();

        in.close();

        System.out.println("Loaded Student Details:");

        for (Student s : loaded) {
            System.out.println("ID: " + s.id);
            System.out.println("Name: " + s.name);
```

```
            System.out.println("Password: " + s.password);
        }

    } catch (Exception e) {
        System.out.println(e);
    }
  }
}
```

Output:

Objects saved successfully.

Loaded Student Details:

ID: 1

Name: Tirth

Password: null

ID: 2

Name: Mann

Password: null

Program 2: Log Analyzer Using Path and Files

Aim:

To develop a log analyzer using Java NIO Path and Files to read multiple text files, count total lines and keyword occurrences, and display the size of each file.

Code:

```
import java.nio.file.*;
```

CEUE203
25IT037

```
import java.io.IOException;
import java.util.*;

public class LogAnalyzer {
    public static void main(String[] args) {

        try {
            Path file1 = Paths.get("log1.txt");
            Path file2 = Paths.get("log2.txt");
            Path file3 = Paths.get("log3.txt");

            Files.write(file1, Arrays.asList(
                "Application started",
                "Error found",
                "User logged in"
            ));

            Files.write(file2, Arrays.asList(
                "Warning detected",
                "Error occurred",
                "System running"
            ));

            Files.write(file3, Arrays.asList(
                "User logged out",
                "Error in database"
            ));
```

```
        Path[] files = {file1, file2, file3};

        int totalLines = 0;
        int keywordCount = 0;

        String keyword = "Error";

        for (Path file : files) {

            System.out.println("File: " + file);

            long size = Files.size(file);
            System.out.println("Size: " + size + " bytes");

            List<String> lines = Files.readAllLines(file);

            for (String line : lines) {
                totalLines++;

                if (line.contains(keyword)) {
                    keywordCount++;
                }
            }
        }

        System.out.println("Total Lines: " + totalLines);
        System.out.println("Keyword Count: " + keywordCount);
```

```
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output:

File: log1.txt

Size: 51 bytes

File: log2.txt

Size: 51 bytes

File: log3.txt

Size: 42 bytes

Total Lines: 8

Keyword Count: 3

Program 3: Folder Tree Report Using NIO

Aim:

To implement Java NIO folder traversal using Files.walk() and generate a report containing file names, sizes and last modified dates in a text file.

Code:

```
import java.nio.file.*;
import java.nio.file.attribute.*;
import java.io.IOException;
import java.util.stream.Stream;
```

CEUE203
25IT037

```
import java.util.*;


public class FolderReport {
    public static void main(String[] args) {


        Path folder = Paths.get(".");


        Path report = Paths.get("folder_report.txt");


        List<String> reportLines = new ArrayList<>();


        try (Stream<Path> paths = Files.walk(folder)) {


            paths.filter(Files::isRegularFile)
                 .forEach(path -> {


                try {
                    BasicFileAttributes attr =
                        Files.readAttributes(
                            path, BasicFileAttributes.class);


                    String details =
                        "Name: " + path.getFileName()
                        + " | Size: " + attr.size()
                        + " bytes | Modified: "
                        + attr.lastModifiedTime();


                    reportLines.add(details);
```

```
                System.out.println(details);

            } catch (IOException e) {
                System.out.println(e.getMessage());
            }
        });

        Files.write(report, reportLines);

        System.out.println("Report generated successfully.");

    } catch (IOException e) {
        System.out.println("Error: " + e.getMessage());
    }
  }
}
```

Output:

Name: log1.txt | Size: 51 bytes | Modified: 2026-10-30T...

Name: log2.txt | Size: 51 bytes | Modified: 2026-10-30T...

Name: log3.txt | Size: 42 bytes | Modified: 2026-10-30T...

Report generated successfully.